

End-to-End Digital-Marketing Product Audit (samujjwal/ghatana commit 4431520ab590)

1. Executive Summary

This report presents an independent, evidence-based audit of the **Digital-Marketing Operating System (DMOS)** contained under `products/digital-marketing` in the `samujjwal/ghatana` repository at commit `4431520ab5902d5b306088aec1cdcde8e4477e22`. The audit follows the ultra-strict checklist for correctness, completeness, UI/UX, backend, database, observability, security, and production readiness. External market research provides context for 2026 digital-marketing trends—AI has become the “operating system” for marketing, requiring end-to-end automation with human approval only for clearly defined risk points ¹; ROI accountability is paramount ²; trust and purpose must be woven into every interaction ³; attention is fragmented across algorithmic channels ⁴; and organisations are restructuring around AI-augmented workflows ⁵. These insights guide the audit’s evaluation of DMOS.

High-level verdict – The latest DMOS implementation shows substantial progress over the earlier backend-only state. It now includes a UI shell, approval workflow, AI action log, command/workflow primitives, Google Ads connector scaffolding, a persistence module, and observability hooks. However, it is **not yet production-ready**. There are mismatched API contracts, incomplete UI-to-backend wiring, placeholder implementations behind feature flags, insufficient coverage thresholds, inconsistent type definitions, and critical gaps in privacy, security, and full-funnel execution. Kernel plugins and boundary hardening tasks are only partially addressed.

Ratings

Area	Rating	Rationale
Correctness	🟡 Partial	Core domain logic for campaign/strategy/proposal workflows is well-structured, but several flows are miswired or stubbed. Approval API responses do not match UI expectations. Google Ads connector throws <code>UnsupportedOperationException</code> .
Completeness	🟡 Partial	Many MVP capabilities (e.g., landing page publishing, CRM-lite, analytics, attribution, rollback, kill switch, lead capture) remain unimplemented or stubbed behind feature flags. UI screens exist but are incomplete and misaligned with backend contracts.
Production readiness	🔴 Not yet	Build gates are violated (hardcoded exceptions), UI uses outdated API contracts, persistence layer lacks migrations, observability is limited to structured log metrics, and tests rely on in-memory repositories.

Area	Rating	Rationale
UI/UX	 <i>Needs work</i>	A dashboard and approval pages exist, but UX is brittle. Required headers (tenant, principal, correlation) are not sent; hooks violate the rules of hooks; roles default to permissive when undefined; forms lack validation; and the dashboard calls hooks after conditional returns.
AI Integration	 <i>Deterministic scaffold</i>	Strategy/budget/proposal generators are deterministic and not model-driven. There is no Agent Orchestrator, prompt versioning, evidence capture, or AI evaluation harness.
Observability (O11y)	 <i>Baseline only</i>	Logs and audit events exist. Metrics collector logs KPI lines but lacks OpenTelemetry traces/metrics and no dashboards. Errors are logged but not correlated across services.
Security & Privacy	 <i>High risk</i>	Contact and suppression models store raw email addresses without hashing; UI does not propagate headers; roles/permissions checks are inconsistent; and there is no retention or DSR workflow.
Kernel alignment	 <i>Partial</i>	Kernel bridge and domain packs are used correctly, but plugin startup wiring, context propagation, and boundary consolidation are incomplete. Risk and compliance plugins remain product-specific; ledger/consent cross-domain ports are missing.

Top P0 issues:

1. **Approval API/UI mismatch** – backend returns `{ items: [...] }` while the UI expects an array of `ApprovalRequest`; approval enums differ; required headers are not propagated.
2. **Production-reachable stubs** – Google Ads connector throws `UnsupportedOperationException` when the feature flag is off; several F2 capabilities are stubbed behind flags without proper gating.
3. **Header propagation & auth** – UI client omits required `X-Tenant-ID`, `X-Principal-ID`, roles, permissions, correlation, and idempotency keys, causing backend to reject requests.
4. **Raw PII storage** – Contact and suppression models persist unhashed email addresses, violating privacy requirements.
5. **Rule-of-hooks violation** – `DashboardPage` conditionally calls hooks after early returns, leading to runtime errors.

Top P1 issues:

1. **Coverage thresholds below target** – modules enforce 85-95 % line coverage, not 100 % for changed code. Many integration tests use in-memory repositories.
2. **UI role handling** – roles default to allow access when list is empty; concurrency safety and error handling are missing.
3. **Incomplete O11y** – metrics are logged rather than exported via OpenTelemetry; correlation IDs are not propagated across modules.

4. **Inconsistent documentation** – `README` files and API docs are stale relative to actual modules and endpoints.
5. **Kernel plugin generalisation** – risk/compliance plugins remain DMOS-specific; boundary resolvers are duplicated across modules.

Recommendation: **Do not release to production** until P0 issues are resolved and P1 issues are addressed. Keep features behind default-off flags and ensure tests cover all flows.

2. Scope & Method

Scope

This audit examines the `products/digital-marketing` subtree of `samujjwal/ghatana` at commit `4431520ab590` along with related modules: UI, application services, domain logic, kernel bridge, domain packs, infrastructure, persistence, connectors, and tests. It also reviews the canonical architecture documents and kernel enhancement plans to understand intended behavior and kernel boundaries. External research on 2026 marketing trends informs the expected capabilities and differentiation ¹ ² . The audit excludes unrelated products in the monorepo and compiled build artifacts.

Method

1. **Inventory** – built inventories of UI routes/components, user actions, backend endpoints/handlers, database models, and tests to ensure nothing is overlooked.
2. **Expected behavior mapping** – used architecture docs, vision docs, and market research to infer the desired product behavior and compare with implementation.
3. **Traceability** – traced each capability from requirement → UI → API → service → domain → persistence → tests.
4. **End-to-end journey simulation** – manually followed flows for strategy creation, approval submission, and dashboard display, verifying data flow and error handling.
5. **Security & privacy analysis** – scanned code for PII handling, authorization checks, feature flags, and potential secrets exposures.
6. **Observability & performance review** – reviewed logs, metrics, traces, and concurrency patterns to assess diagnosability and scalability.
7. **Kernel integration review** – cross-checked plugin usage with kernel decision guides and boundary hardening plan to identify alignment gaps ⁶ ⁷ .
8. **Market research** – studied public reports on 2026 marketing trends to validate DMOS's strategic direction: AI-native operations, ROI measurement, trust/purpose, algorithmic discovery, and AI-augmented organisations ¹ ³ .

3. Product Inventory

3.1 UI Inventory

The UI is implemented in `ui/` (React/Vite). Key routes and components:

UI Item	File(s)	Purpose	User Actions	Data Dependencies	API/State
DashboardPage	ui/src/pages/ DashboardPage.tsx	Landing dashboard summarising growth goal, approvals, AI actions, workflow status, risk/compliance, and metrics	Navigates after login; loads approval queue and AI action log	Requires tenant/principal from AuthContext; fetches pending approvals and AI actions	useApp, useAi, getPe, and g, API call
ApprovalQueuePage	ui/src/pages/ ApprovalQueuePage.tsx	Lists pending approval requests; allows approval/rejection	Click approve/decline; filter by target type; view details	Depends on useApprovalQueue hook; uses ApprovalList and ApprovalCard components	listP, API; d, useNo
ApprovalDetailPage	ui/src/pages/ ApprovalDetailPage.tsx	Shows approval snapshot detail and allows decision	Approve/decline with justification	useApprovalDetail hook; uses ApprovalDetail component	getAp, and d, state; u, for con
AiActionLogPage	ui/src/pages/ AiActionLogPage.tsx	Displays AI-driven actions (automations, prompts)	Filter logs, view details	useAiActionLog hook	listA

UI Item	File(s)	Purpose	User Actions	Data Dependencies	API/State
LoginPage/ AuthProvider	ui/src/pages/ LoginPage.tsx , AuthContext.tsx	Handles login and stores token, tenant, workspace, roles	Login form; token exchange; context provider	auth/login API; local storage	Partial

3.2 User Action Inventory

Key user actions include: login, create strategy, submit strategy for approval, approve/decline requests, view AI action log, view dashboard metrics, and trigger AI suggestions. Each action is summarised below:

Action	UI Source	Expected Result	Handler	Backend/API	DB Impact	State
Login	LoginPage	User enters credentials, receives token, tenant/workspace context	handleLogin function	auth/login endpoint	None; token stored in local storage	Re da
Generate Strategy	StrategyPage (missing)	User fills brief → AI generates marketing strategy; user can edit and submit for approval	No UI implemented yet	strategy/generate API (deterministic generator)	Creates Strategy aggregate in DB	Dis pre all su
Submit Strategy for Approval	StrategyPage (missing)	Converts user's draft into approval request requiring human sign-off	None	approval/submit API; strategy/submit API	Creates ApprovalRequest in DB; logs audit event	Re to qu sh su toa

Action	UI Source	Expected Result	Handler	Backend/API	DB Impact	Success State
Approve/ Decline Request	ApprovalQueuePage , ApprovalDetailPage	Approver selects a request, enters comment, approves or declines	decide function in hooks	approval/decide API	Updates Approval entity, triggers workflow	Shows up status up date me
View AI Action Log	AiActionLogPage	Shows list of AI-initiated actions (e.g., agent runs, prompts, outputs)	useAiActionLog hook	aiAction/list API (stub)	Reads from event store	Re with tim an

3.3 Backend/Service Inventory

Key backend modules and their responsibilities:

Backend Item	File(s)	Caller(s)	Dependencies
CampaignServiceImpl	dm-application/src/main/java/.../ CampaignServiceImpl.java	Campaign API, Workflow engine	Campaign API, Workflow engine
StrategyServiceImpl	dm-application/src/main/java/.../ StrategyServiceImpl.java	Strategy API	Strategy API

Backend Item	File(s)	Caller(s)	
ApprovalServiceImpl	<code>dm-application/src/main/java/.../ApprovalServiceImpl.java</code>	Approval API, UI	
CommandServiceImpl	<code>dm-application/src/main/java/.../DmCommandServiceImpl.java</code>	Workflow engine	
WorkflowServiceImpl	<code>dm-application/src/main/java/.../DmWorkflowServiceImpl.java</code>	Workflow engine	
GoogleAdsCampaignConnectorServiceImpl	<code>dm-connector-google-ads/src/main/java/.../DmGoogleAdsCampaignConnectorServiceImpl.java</code>	Command service	

3.4 Database Inventory

The `dm-persistence` module defines JPA entities and repository interfaces. Notable models:

DB Item	File(s)	Purpose	Callers	Co
CampaignEntity	dm-persistence/src/main/ java/.../ CampaignEntity.java	Stores campaigns with id, tenant, workspace, name, budget, state, target URL, audit metadata	CampaignRepository	Un con on wo na no id, sta
StrategyEntity	.../StrategyEntity.java	Stores strategies with id, plan JSON, budget recommendation, status	StrategyRepository	Un con (te wo ven no fie
ApprovalEntity	.../ApprovalEntity.java	Stores approval requests with id, target type, target id, status, approver roles, required role, snapshot JSON, submittedBy, submittedAt	ApprovalRepository	Un inc en tar
CommandEntity, WorkflowEntity	.../CommandEntity.java , WorkflowEntity.java	Stores commands and workflow states	Services above	Un for to wo en sta
ContactEntity, SuppressionEntryEntity	.../ContactEntity.java , SuppressionEntryEntity.java	Stores leads/ contacts and suppression lists	ContactRepository , SuppressionRepository	Un em en con po

3.5 Test Inventory

Tests reside under `dm-domain`, `dm-application`, `dm-api`, `dm-integration-tests`, `dm-connector-google-ads`, and `ui`.

Test	File	Type	Feature Covered	What It Proves	Real
CampaignServiceImplTest	<code>dm-application/src/test/.../CampaignServiceImplTest.java</code>	Unit	Campaign creation, launch, pause, complete, and error cases	Validates domain transitions, audit calls, compliance checks	Uses <code>dm-domain</code> <code>dm-application</code> <code>dm-api</code> <code>dm-integration-tests</code> <code>dm-connector-google-ads</code> <code>ui</code>
StrategyServiceImplTest	<code>dm-application/src/test/.../StrategyServiceImplTest.java</code>	Unit	Strategy generation and submission	Asserts deterministic generator output, audit events	In-m <code>dm-domain</code> <code>dm-application</code> <code>dm-api</code> <code>dm-integration-tests</code> <code>dm-connector-google-ads</code> <code>ui</code>
ApprovalServiceImplTest	<code>dm-application/src/test/.../ApprovalServiceImplTest.java</code>	Unit	Approval submission, decision, status retrieval	Tests status transitions and validation	In-m <code>dm-domain</code> <code>dm-application</code> <code>dm-api</code> <code>dm-integration-tests</code> <code>dm-connector-google-ads</code> <code>ui</code>
WorkflowServiceImplTest	<code>dm-application/src/test/.../WorkflowServiceImplTest.java</code>	Unit	Workflow lifecycle transitions	Asserts transitions; role gating; list/filter	In-m <code>dm-domain</code> <code>dm-application</code> <code>dm-api</code> <code>dm-integration-tests</code> <code>dm-connector-google-ads</code> <code>ui</code>
ApiIntegrationTest	<code>dm-integration-tests/src/test/...</code>	API E2E (ActiveJ server)	Tests strategy and campaign endpoints using HTTP	Spins up ActiveJ server with in-memory modules	Valid <code>dm-domain</code> <code>dm-application</code> <code>dm-api</code> <code>dm-integration-tests</code> <code>dm-connector-google-ads</code> <code>ui</code>

Test	File	Type	Feature Covered	What It Proves	Real
UI unit tests	ui/src/tests	Component tests	Components render, handle basic props	Asserts snapshots and prop mapping	React Vites API
Playwright E2E tests	ui/playwright	Browser E2E	End-to-end flows for login and approval	Launches UI; uses stubbed API endpoints	Valid

Coverage remains below 100 % for many modules. Many flows (AI actions, Google Ads, workflows) lack tests entirely.

4. Product Behavior Map

The intended DMOS behavior (derived from architecture docs and market research) can be summarised as follows:

Capability	User/ Persona	Problem Solved	Expected UX	Expected Backend Behavior	Expected Data Behavior	Success Criteria
Self-marketing engine	Small business owners	Need to acquire customers without marketing expertise	Dashboard displays growth goal and next best action; AI recommends strategy; user approves or tweaks	Agent orchestrator generates strategy, budget, copy, landing page; compliance preflight; workflows execute commands; analytics feed back into plan	Domain models for strategy, campaign, content, contact, approval; events store actions; workflows and commands persist; data anonymised	User sees leads generated and revenue vs. cost; campaign is compliant; minimal manual steps

Capability	User/ Persona	Problem Solved	Expected UX	Expected Backend Behavior	Expected Data Behavior	Success Criteria
Strategy generation	Business owner	Need a marketing plan	Brief wizard collects goals; AI returns strategy with channels, assets, schedule	Service calls agent orchestrator; queries models; returns deliverables; logs audit; persists plan	Strategy stored with versions; deliverables parsed; updates track modifications	Plan matches user intent; deliverables realistic; preflight passes
Proposal/SOW & approval	Business owner & approver (manager/ lawyer)	Need to convert plan into contract and sign off	Proposal page shows deliverables, budget, timeline; user can sign or request changes; approvals queue shows pending requests	Service compiles deliverables; calculates pricing options; triggers approval workflow; logs audit; respects roles	Proposal stored; approval snapshot persisted; event triggers further tasks; SOW stored for contract	Proposal accepted/ modified; approval captured; compliance conditions satisfied
Campaign execution	DMOS system & connectors	Need to run campaigns across channels	Dashboard shows active campaigns; user can pause/ resume; system auto-executes tasks; user sees metrics	Service converts plan into commands; outbox triggers connectors; idempotency ensures no duplicates; events record results	Commands persisted; workflows track state; metrics aggregated; budgets updated	Campaign runs automatically; budgets spent correctly; leads captured; user can pause or kill

Capability	User/ Persona	Problem Solved	Expected UX	Expected Backend Behavior	Expected Data Behavior	Success Criteria
Lead capture & CRM-lite	System	Need to collect leads	Landing pages and forms auto-created; user sees leads list; AI scores leads	Connectors gather leads; store in Contact entity; scoring engine ranks; suppression lists updated; consent recorded	Contacts stored with hashed identifiers; suppression maintained; consent proof stored	Leads captured; duplicates suppressed; compliance satisfied
Analytics & optimisation	System & user	Need to measure ROI and adjust strategy	Dashboard shows metrics, ROI, micro-conversions; AI suggests optimisations; user can accept/ reject	Analytics engine aggregates events; attribution models compute ROI; agent orchestrator suggests experiments; logs actions	Aggregated metrics stored; personal data anonymised; experiments recorded	ROI tracked; user receives actionable insights; improvements enacted
Governance & compliance	System & approvers	Need to ensure marketing complies with laws & brand guidelines	Preflight warnings, required approvals, transparency log	Compliance engine runs rules; approval service enforces roles; consent management; risk engine flags issues	All actions audited; snapshots stored; user identity hashed; retention rules applied	No campaign or proposal violates compliance; audit log complete; data subject rights honoured

5. Requirement-to-Implementation Traceability

The following highlights major requirements and traces them to implementation:

Requirement / Capability	UI Route/Page	User Actions	API/Backend Handler	Service/Domain Logic	D
Strategy generation from brief	Missing (brief wizard not implemented)	None	<code>strategy/generate</code>	<code>StrategyServiceImpl.generate</code> uses deterministic generator	S JS
Proposal & SOW generation	Not in UI; <code>ProposalPage</code> absent	None	<code>proposal/generate</code> , <code>sow/generate</code> (present)	<code>ProposalServiceImpl</code> constructs deliverables and pricing; <code>SowServiceImpl</code> compiles tasks and schedule	P S
Submit for approval	<code>ApprovalQueuePage</code> (not integrated with strategy)	Approve/decline button	<code>approval/submit</code> , <code>approval/decide</code> , <code>approval/status</code>	<code>ApprovalServiceImpl</code> stores snapshot, validates, triggers workflow	A
Launch campaign (Google Ads)	Launch button in UI (future)	None	<code>campaign/launch</code>	<code>CampaignServiceImpl.launch</code> changes state and triggers connector command	C C
Dashboard metrics	<code>DashboardPage</code>	None; automatically displays	<code>metrics/list</code> (not implemented)	<code>DmosMetricsCollector</code> logs metrics to logs; no API	N
AI action log	<code>AiActionLogPage</code>	None	<code>aiAction/list</code> (stub)	Not implemented	A (M)
Contact/consent management	UI absent (public intake missing)	None	<code>contact/intake</code> , <code>consent/record</code> (missing)	Domain objects exist but not exposed	C C
Suppression lists	None	None	<code>suppression/add</code> (missing)	Domain object exists	S

Requirement / Capability	UI Route/Page	User Actions	API/Backend Handler	Service/Domain Logic	D
Lead scoring & CRM lite	None	None	lead/score (missing)	LeadScoringServiceImpl (absent)	L
Observability (metrics/traces)	None	None	N/A	DmosMetricsCollector logs metrics; DmKernelBridge.recordAudit logs events	A

6. End-to-End Journey Audit

6.1 Strategy Creation & Approval Journey

Entry point: (Missing) – There is no strategy wizard UI; user cannot collect goals or create strategy.

Expected: User enters business goals → AI generates strategy with channels, budget, deliverables → user tweaks → submits for approval → approver reviews and approves or requests changes → campaign is created.

Actual: Only backend endpoint exists; strategy generation is deterministic; UI absent; approval snapshot fields do not match UI; there is no path to propose deliverables or budget; no way to sign SOW. Snapshots include raw plan JSON, but there is no versioning, editing, or AI explanation. Tests do not cover this journey.

Gaps: Missing UI wizard; no AI generation; no editing flow; missing approval integration; preflight warnings not shown; no concurrency or idempotency. Severity: P0.

6.2 Campaign Launch Journey

Entry point: (Future) – Launch button expected on dashboard or campaign page.

Expected: User selects strategy → triggers campaign launch → system converts plan into commands (e.g., create Google Ads campaign, publish landing page, send emails) → compliance preflight ensures safe content → connectors execute commands with retries → system tracks progress → user can pause/resume/kill.

Actual: CampaignServiceImpl.launch toggles internal state only. DmCommandServiceImpl and DmWorkflowServiceImpl exist but connectors throw UnsupportedOperationException if feature flag disabled; outbox/inbox pattern is missing; there is no event registry or durable workflow execution. UI has no launch button. Tests only verify state transition.

Gaps: Execution not implemented; Google Ads connector stub; no error handling; no idempotency; no rollback/kill switch; no analytics; no UI. Severity: P0.

6.3 Approval Journey

Entry point: `ApprovalQueuePage` on UI loads pending approvals.

Expected: Approver sees pending requests with summary (target, type, risk level, requester, time) → selects request → views snapshot → decides to approve or decline with comment → backend updates status and triggers subsequent workflows; UI updates accordingly.

Actual: UI fetches `listPendingApprovals` but expects array; backend returns `{ items: [...] }`, causing UI not to render. UI's `ApprovalRequest` type differs from backend; fields like `workspaceId`, `riskLevel`, `requiredApproverRole` do not exist in backend. Approval decisions call `decide` endpoint without required headers; backend rejects with unauthorized error. Approver role check in UI defaults to permissive if roles array is empty. Tests stub the API, so this mismatch is not caught.

Gaps: API contract mismatch; header propagation missing; role handling insecure; concurrency & error states not handled; missing pagination. Severity: P0.

6.4 Dashboard Journey

Entry point: `DashboardPage` accessible after login.

Expected: Dashboard shows growth goal, current campaigns, approvals needing action, AI actions, risk/compliance warnings, metrics (spend vs. ROI), and next best actions. It should gracefully handle loading, empty, error, and success states; provide context links; and support multi-tenant filtering.

Actual: Dashboard shows placeholders; uses hooks to fetch approvals and AI actions but does not handle errors; calls hooks after conditional returns; does not pass required headers; misinterprets API response shape; no metrics or next best actions; loading and empty states are not implemented.

Gaps: Implementation is incomplete; fails to comply with low cognitive load and clarity; missing O11y instrumentation and analytics. Severity: P1.

7. UI/UX Audit

Key findings:

Area	Files	Finding	Correctness Impact	Completeness Impact	Severity	Risk
Hook ordering	<code>DashboardPage.tsx</code>	Calls <code>useAiActionLog</code> after conditional return; violates React rules	Runtime error; unpredictable state	Prevents proper loading of AI actions	P0	M to critical if

Area	Files	Finding	Correctness Impact	Completeness Impact	Severity	
API response mismatches	apiClient.ts , useApprovalQueue.ts	UI expects ApprovalRequest[] , backend returns { items: [...] } ; fields mismatch	UI fails to render approvals	Incomplete approval flow	P0	A d u e U in te
Header propagation	apiClient.ts	Does not send X-Tenant-ID , X-Principal-ID , X-Roles , X-Permissions , correlation ID, idempotency key	Backend rejects requests; fails auth	User cannot perform actions	P0	P re in re A a re h g c p
Role handling	useAuth.tsx , ApprovalQueuePage.tsx	Approver list defaults to allow when roles array is empty	Approvers may see actions they are not authorized for	Security risk	P1	If U e u p e c s in
Validation & error states	various forms	Forms do not validate inputs (e.g., login, approval comment) or handle backend errors	Misleading states; user sees blank pages or generic errors	User frustration	P1	A v d fi e b a a m

Area	Files	Finding	Correctness Impact	Completeness Impact	Severity	R
Responsive design	UI components	Layout uses basic grid; not tested on mobile	Potential breakage on small screens	Incomplete UX	P2	In re d C fr te b
Accessibility	UI components	Missing ARIA labels; modals not focus-trapped; contrast issues untested	Accessibility barriers	Incomplete compliance	P2	A la in fo m te a to

8. Frontend Action & State Audit

Key flows were inspected for correctness; major issues include:

- **Duplicate submissions** – `ApprovalDetailPage` allows repeated clicks on approve/decline without disabling button; backend may process duplicates. Add idempotency keys and disable button during request.
- **Optimistic updates** – UI does not optimistically update approval list; page remains stale until manual refresh. Add stale-while-revalidate pattern and event stream.
- **Error swallowing** – API hooks catch errors but only log to console; error messages are not surfaced to user. Use toast or inline error states.
- **State duplication** – Role and permission state are stored in both context and local state; unify into a single context provider.
- **Cache invalidation** – After approval decision, `useApprovalQueue` does not revalidate; list remains outdated. Use SWR or query invalidation triggers.

9. Backend/API/Domain Audit

Major issues:

- **UnsupportedOperationException** – Google Ads connector throws `UnsupportedOperationException` in production code, violating quality gates. Replace with feature flag gating that returns a descriptive error and ensure the feature is disabled in production until ready.
- **Idempotency** – Command and workflow services lack idempotency tokens; repeated triggers could cause duplicate external actions. Add idempotency keys and repository constraints.

- **Error handling** – Many services catch `Exception` and rethrow as generic runtime exceptions; no specific error shapes or status codes returned to API clients.
- **Feature flags** – Incomplete features (e.g., AI action log, analytics) are partially exposed; ensure flags default off in production and UI hides disabled features.
- **Domain rules** – Preflight checks for claims, disclosures, consent, and risk are defined but not enforced at runtime; integrate compliance plugin results into service decisions.
- **Audit logging** – Audit events are recorded but may contain raw PII; redaction and data minimisation should be applied.

10. Database & Data Integrity Audit

Key concerns:

- **PII storage** – `ContactEntity` and `SuppressionEntryEntity` store raw email addresses and phone numbers. The canonical design expects hashed contact points with salt; fix this by storing hashed identifiers and storing raw PII only in encrypted form.
- **Migrations** – No schema migrations are provided for MySQL/PostgreSQL; only JPA definitions exist. Add Flyway or Liquibase migrations and versioning.
- **Unique constraints** – Some unique constraints (e.g., `StrategyEntity` version) do not enforce uniqueness across tenants; consider composite keys.
- **Referential integrity** – Entities store IDs as strings without foreign key constraints (e.g., `workflowId` in `CommandEntity`); add constraints.
- **Retention & soft delete** – Entities lack TTL or archival policies; add expiration columns and scheduled jobs to purge old data.

11. Production Mock/Stub Audit

Scanning the code for mocks, stubs, placeholders, TODOs, and test utilities reveals:

File	Evidence	Production Reachable?
<code>DmGoogleAdsCampaignConnectorServiceImpl.java</code>	throw new UnsupportedOperationException("Google Ads connector disabled")	Yes if feature flag disabled

File	Evidence	Production Reachable?
<code>DmosMetricsCollector.java</code>	<code>// TODO: wire to MetricCollectorPort when available</code>	Yes
<code>AiActionLogPage.tsx</code>	Placeholder UI; returns empty list; console logs	Yes
Domain pack rules referencing <code>TODO</code>	Comments indicating new rules to implement (e.g., claim validations, risk scoring)	Yes
In-memory repositories used in integration tests	<code>InMemoryCampaignRepository</code> etc.	No (test only)

12. Duplicate/Source-of-Truth Audit

Examples of duplication:

- **Approval enum definitions** – UI defines `ApprovalTargetType` with values (`CAMPAIGN` , `CONTENT` , etc.), while backend defines `DmApprovalTargetType` (`STRATEGY` , `PROPOSAL` , `SOW` , `CONTENT_VERSION` , `BUDGET` , `CAMPAIGN_LAUNCH` , etc.). Unify definitions in a shared package and generate TypeScript enums from Java.
- **Role names** – UI hardcodes `ROLE_MANAGER` , `ROLE_ADMIN` , etc., while backend uses `DmRole` enum. Create a shared roles definition to avoid drift.
- **API URL strings** – Both UI and backend tests define endpoints manually; unify through an OpenAPI spec or shared constants.
- **Hooks and state logic** – Multiple contexts store authentication data; unify into a single provider.
- **Compliance rules** – Duplicate rule classes exist across domain packs; consolidate into a single rule engine and share across products ⁷ .

13. Security, Privacy, Governance, Permission Audit

Findings:

Area	File(s)	Risk	Expected Behavior	Actual Behavior	Severity	Refractor
Tenant & user isolation	API controllers (<code>DmCampaignServlet</code> , etc.)	Without headers, backend cannot verify tenant/principal; UI doesn't send them	All requests must include <code>X-Tenant-ID</code> , <code>X-Principal-ID</code> , roles, permissions; kernel must enforce isolation	API accepts requests with missing headers; in tests, allow-all kernel used; potential cross-tenant data access	P0	En... m... h... re... w... m... u... to... th...
Role/permission checks	<code>ApprovalServiceImpl</code> , UI hooks	Approver list defaults to true when roles array empty	Should deny when user lacks required role	UI and backend treat empty role list as allow; risk of unauthorized approvals	P1	In... m... ro... n... b... m... e... re... ro... a... cr...
PII hashing	<code>ContactEntity</code> , <code>SuppressionEntryEntity</code>	Raw email/phone stored	Should hash contact points and store raw values encrypted; use reversible tokens only for deliverability	Entities store raw email; no encryption; risk of breach	P0	H... co... p... se... sa... en... ra... u... q... u... h... ke... in... D... d... re...

Area	File(s)	Risk	Expected Behavior	Actual Behavior	Severity	Re Fi
Consent management	Consent proof not enforced	Should require consent before adding contact to campaign; store proof with timestamp and origin	Not implemented; service accepts contact without consent	P1	Integrate consent plugin; add API to record consent; block marketing actions without valid consent	W ve co re an re
Secrets management	Google Ads API keys & tokens	Hardcoded placeholder values; secrets may appear in logs	Should use secret manager; never log secrets	Implementation stub uses config file; may log tokens on error	P1	In w m m se lo te en se lo
Audit & retention	AuditEntity, DmosMetricsCollector	Audit events recorded but include raw user identifiers; retention unspecified	Should record minimal necessary info; apply retention schedule; allow DSAR deletion	Raw PII may be logged; no TTL on audit logs	P1	R a re p in D en

14. Observability & Operability Audit

Observability is limited to logs and simple metrics:

Flow	Logs	Metrics	Traces	Audit Events	Gaps
Strategy generation	Logs via <code>DmosObservabilityBaseline</code> show start/end; audit events recorded	Metrics recorded via <code>DmosMetricsCollector</code> (structured logs)	No distributed traces; correlation ID not propagated	Yes (audit)	Lacks OTel spans; metrics not exported; no dashboards
Approval decision	Logs actions; audit events recorded	Decision metrics logged	No traces	Yes	Missing traces and metrics for failure/success latency; no alerting on high pending approvals
Campaign launch	Logs command creation; audit events recorded	No metrics for execution success/failure; no trace across connectors	Audit event recorded but no context	Missing metrics; no visibility into Google Ads call success; no DLQ metrics	Add command execution metrics; trace across connector; record DLQ counts; integrate with kernel event registry
Workflow engine	Logs state changes; metrics logged	No metrics for queue sizes, retries, failures; no trace across workflows	Audit events recorded only for manual steps	Missing queue metrics; cannot diagnose stuck workflows	Instrument queue size, processing time, retries; add job monitoring dashboard

15. Performance & Scalability Audit

Concerns:

- **ActiveJ event loop blocking** – Google Ads connector wraps blocking HTTP call with `Promise.ofBlocking(() -> httpClient.execute(...))` but passes `Runnable::run`, which runs in calling thread. This blocks the event loop and can degrade throughput. Use dedicated worker pool for blocking operations.
- **Database indexing** – Lack of composite indexes (tenant/workspace) can cause table scans; add indexes on frequently queried fields.
- **Pagination** – List endpoints return full lists without cursor or page size; may degrade performance with many records; implement `limit` and `cursor` parameters.
- **Bundle size** – UI does not employ code splitting; entire app may load at once; implement dynamic imports.
- **Workflow concurrency** – Without concurrency control and idempotency tokens, command execution may race; implement concurrency locks or atomic state transitions.

16. Test Audit

Major test gaps:

Capability/ Flow	Existing Tests	Missing Tests	Invalid Tests	Required Tests	Priority
Strategy & proposal flows	<code>StrategyServiceImplTest</code> (unit)	UI test for strategy wizard; integration test for generate/ submit/ approve; negative tests for invalid input; concurrency tests	None	Browser E2E for strategy creation & approval; API E2E with DB	P0

Capability/ Flow	Existing Tests	Missing Tests	Invalid Tests	Required Tests	Priority
Approval flow	Unit tests cover basic transitions	Test for API/ DTO shape mismatch; multi-page pending list; high load; unauthorized access; concurrency; UI integration	Approval UI tests rely on stubbed API; may mask contract mismatch	API integration tests verifying list returns correct shape; UI tests verifying miswired headers; concurrency tests with multiple approvers	P0
Campaign launch & connector	Unit test covers state transition only	End-to-end test launching a campaign with connector enabled; error handling on failure; idempotency; DLQ; rollback	None	Integration tests with real/ Mock Google Ads service verifying API calls; tests for outbox/inbox pattern; workflow rollback	P0
Persistence & migrations	None	DB integration tests verifying constraints, indexes, hashed PII; migration tests to ensure compatibility	N/A	Integration tests with real DB; test migrations up/ down; test hashed data storage	P0

Capability/ Flow	Existing Tests	Missing Tests	Invalid Tests	Required Tests	Priority
Observability & O11y	None	Tests verifying metrics emission, logs redaction, correlation ID propagation, OTel spans	N/A	Unit tests capturing logs; integration tests verifying metrics exported; instrumentation tests	P1
Performance/ load	None	Load test for approval queue under 1000 requests; stress test for workflow engine; concurrency test	N/A	JMH or Gatling tests; test memory usage; test throughput; define SLOs	P2

17. Prioritised Remediation Plan

Priority	Area	Issue	Evidence
P0	Approval Flow	API response shape mismatch; UI missing headers; roles default to permissive	<code>DashboardPage.tsx</code> , <code>ApprovalServiceImpl.j</code>
P0	Execution & Connector	Google Ads connector throws <code>UnsupportedOperationException</code> in production code	<code>DmGoogleAdsCampaignConnectorServiceImpl.</code>

Priority	Area	Issue	Evidence
P0	Privacy	Raw PII storage in contacts and suppression lists	ContactEntity.java , SuppressionEntryEntity.java
P0	Idempotency & Concurrency	Commands and workflows lack idempotency tokens; duplicate actions possible	DmCommandServiceImpl.java , WorkflowServiceImpl.java
P0	UI Hook ordering	Hooks called conditionally causing runtime error	DashboardPage.tsx
P1	Metrics & O11y	Metrics logged but not exported; no OpenTelemetry instrumentation	DmosMetricsCollector.java , DmosObservabilityBaseline.java
P1	Documentation & APIs	Stale README.md and API docs; mismatch with code; missing OpenAPI spec	products/digital-marketing/README.md , dm
P2	Duplicate definitions	Approval enums, roles, endpoints defined separately in UI/backend	ApprovalTargetType.ts , DmApprovalTargetType.java

Priority	Area	Issue	Evidence
P2	Responsive & accessibility design	UI not responsive; no ARIA labels	UI files
P2	Database migrations	Lack of migrations and foreign keys	dm-persistence

18. Production Readiness Gate

Ready for production: No – P0 issues must be resolved and P1 tasks completed. UI/Backend contract mismatches, PII storage, and stubbed connectors make the product unsafe.

Ready for internal demo: Yes, behind strict feature flags. Limit demo to strategy generation (deterministic), approval UI (with stubbed backend), and dashboard placeholders; disable Google Ads connector; hide incomplete features.

Ready behind feature flag: Possibly. Hide F2/F3 features (connectors, AI actions) behind disabled flags; restrict approval flows to non-critical items; do not expose incomplete metrics/analytics.

Critical blockers: API/UI mismatches; stubbed Google Ads connector; raw PII storage; missing headers; rule-of-hooks violations; missing idempotency.

Minimum fixes before release: Fix all P0 tasks above; implement P1 tasks for O11y and documentation.

19. Final Checklist

Before release:

1. **Correctness & completeness** – All flows implemented end-to-end (strategy → proposal → approval → campaign → analytics). Confirm compliance checks and risk assessments are enforced.
2. **No production mocks/stubs** – Remove all `UnsupportedOperationException`, TODOs, placeholders; ensure connectors either disabled with error or fully implemented.
3. **UI/UX** – Dashboard and flows must be responsive, accessible, low cognitive load, with clear primary actions and error states.
4. **Backend/API** – Contracts must match UI; validation and error handling must be complete; idempotency enforced.
5. **DB/data integrity** – Schemas must be migrated; hashed PII; constraints and indexes added; retention policies defined.

6. **Security/privacy** – Tenant isolation, role/permission checks, consent management, DSAR processes, encryption and hashing, secret management.
 7. **Observability** – Add OpenTelemetry instrumentation; propagate correlation IDs; export metrics; create dashboards and alerts.
 8. **Performance** – Fix blocking calls; add indexes; implement pagination and caching; test concurrency.
 9. **Tests** – Achieve 100 % coverage for changed code; add integration tests across API and DB; add UI E2E tests; test negative and edge cases; include load and concurrency tests.
 10. **Documentation** – Update README, architecture docs, API docs, and user guides; provide runbooks; update roadmaps.
-

This audit aims to provide a comprehensive roadmap to transition DMOS from a promising scaffold to a production-ready, AI-native, compliance-first marketing operating system that aligns with 2026 market expectations ¹.

¹ ² ³ ⁴ ⁵ Discover our Marketing Trends of 2026

<https://www.deloittedigital.com/nl/en/insights/perspective/marketing-trends-2026.html>

⁶ `KERNEL_MODULE_EXTENSION_PLUGIN_DECISION_GUIDE.md`

https://github.com/samujjwal/ghatana/blob/4431520ab5902d5b306088aec1cdcde8e4477e22/docs/kernel/KERNEL_MODULE_EXTENSION_PLUGIN_DECISION_GUIDE.md

⁷ `KERNEL_BOUNDARY_HARDENING_PLAN.md`

https://github.com/samujjwal/ghatana/blob/4431520ab5902d5b306088aec1cdcde8e4477e22/KERNEL_BOUNDARY_HARDENING_PLAN.md